ClearPath

Document de produs și implementare (MVP) · Four The Win · Astra Hack, 6-7 noiembrie 2026

1. Ce este ClearPath

ClearPath este o aplicație web care arată cum este experimentat un site de un utilizator nevăzător. Clientul introduce linkul unei pagini, iar ClearPath deschide pagina cu un agent automat, o parcurge exact cum o parcurge un cititor de ecran (element cu element și cu tasta Tab) și generează un clip video narat: pe ecran se vede elementul pe care se află utilizatorul, iar vocea rostește ce ar auzi el. În paralel, aplicația rulează un audit automat de accesibilitate și livrează lista de probleme găsite, fiecare legată de secunda din clip în care apare, un scor de la 0 la 100 și o recomandare de remediere pentru fiecare problemă.

Pe scurt: **un link intră, un clip narat, o listă de probleme și un scor ies.**

2. Cum funcționează

Pas	Ce se întâmplă
1. Trimiterea	Utilizatorul introduce URL-ul. API-ul validează că pagina răspunde, creează o analiză cu ID unic și o pune în coadă. Interfața afișează statusul în timp real.
2. Extracția	Un worker deschide pagina în Chromium controlat de Playwright și extrage arborele de accesibilitate (rol, nume, stare, poziție pentru fiecare element). Face două parcurgeri: liniară (citire de sus în jos) și cu Tab (doar elementele focusabile).
3. Anunțurile	Motorul de anunțuri transformă fiecare element în propoziția pe care o rostește un cititor de ecran, de exemplu „Titlu nivel 1, Finalizează comanda” sau „Buton” pentru un buton fără nume.
4. Auditul	axe-core rulează în pagină și găsește problemele măsurabile. Fiecare problemă este legată de elementul ei din arbore.
5. Clipul	Fiecare propoziție devine audio (Piper TTS). Agentul parcurge pagina din nou, evidențiază elementul curent cât timp se aude propoziția și filmează. ffmpeg lipește audio și video într-un singur MP4 și rezultă momentul exact al fiecărui element.
6. Raportul	Problemele primesc secunda din clip, se calculează scorul și se atașează recomandările. Pagina de rezultat afișează playerul, lista de probleme (click pe problemă = salt în clip) și scorul.

3. Ce intră în MVP și ce nu

Intră

- O singură pagină per analiză (URL public, fără autentificare pe site-ul analizat).
- Clip de maxim ~90 de secunde (implicit primele 40 de anunțuri din parcurgerea liniară).
- Parcurgere cu Tab completă, cu detectarea capcanelor de tastatură.
- Audit axe-core, scor numeric, recomandări din șabloane pe tip de problemă.
- Pagina de rezultat cu player, listă navigabilă și link de partajare.

Nu intră (după hackathon)

- Conturi, login, plăți, istoric de analize.
- Fluxuri pe mai multe pagini (de exemplu produs → coș → checkout).
- Validarea comparativă cu NVDA real.
- Stocare în cloud (S3/MinIO); fișierele stau pe disc local.
- Recomandări generate de LLM (doar ca extra, dacă rămâne timp).

4. Tehnologii

Componentă	Tehnologie	Rol
Limbaj backend	Python 3.11	Un singur limbaj pentru API și worker, integrare mai rapidă
API	FastAPI	Endpoint-uri pentru creare analiză, status, rezultat, fișiere media
Coadă și joburi	Redis + RQ	Analiza durează minute, rulează într-un worker separat
Browser automat	Playwright (Python) + Chromium	Deschide pagina, parcurge cu Tab, filmează
Arbore de accesibilitate	Chrome DevTools Protocol	Accessibility.getFullAXTree, aceeași sursă ca NVDA/VoiceOver
Audit	axe-core (injectat în pagină)	Detectarea problemelor măsurabile WCAG
Sinteză vocală	Piper TTS, voce ro_RO-mihai-medium	Rulează local pe CPU, fără cheie API
Montaj	ffmpeg	Mixare audio peste video, export MP4
Interfață	Next.js + Tailwind	Formular, status, pagina de rezultat
Stocare	Disc local, data/{id}/	Fișierele fiecărei analize într-un folder
Rulare	docker-compose	api, worker, redis, web pornesc cu o comandă

Limba vocii: anunțurile sunt în română (rolurile: „Buton”, „Link”, „Titlu nivel 2”), iar numele elementelor se citesc așa cum sunt pe site, cum face și NVDA setat în română pe un site în engleză. Engleza se poate adăuga ulterior ca a doua listă de șabloane.

5. Arhitectură și structura proiectului

Trei servicii: **web** (Next.js) vorbește cu **api** (FastAPI), care pune joburi în **redis**; **worker**-ul le preia și rulează pipeline-ul. Pipeline-ul este o succesiune de funcții pure, fiecare citește și scrie fișiere JSON în `data/{id}/`. Asta permite ca fiecare etapă să fie scrisă și testată separat, pe date de test.

```
clearpath/
├── contracts/          # ora 1: scheme + date de test comune (sample_*.json, selector.js)
├── worker/
│   ├── extract/        # Membrul 1: extract(url) -> nodes.json
│   ├── engine/         # Membrul 2: build_events(), run_audit(), build_report()
│   ├── media/          # Membrul 3: render_clip(url, events) -> clip.mp4 + timeline.json
│   └── pipeline.py     # Membrul 4: leagă etapele, actualizează statusul
├── api/                # Membrul 4: FastAPI
├── web/                # Membrul 4: Next.js
├── data/{id}/          # nodes.json, events.json, issues.json, timeline.json, report.json, clip.mp4
└── docker-compose.yml  # Membrul 1: imaginea worker (Playwright, Piper, ffmpeg)
```

Ordinea în pipeline:

```
nodes    = extract(url)                          # M1
events   = build_events(nodes)                   # M2
issues   = run_audit(url, nodes)                 # M2
timeline = render_clip(url, nodes, events)       # M3  (scrie și clip.mp4)
report   = build_report(nodes, issues, timeline) # M2
```

6. Contractele de date

Acestea se stabilesc în prima oră și nu se mai schimbă fără acordul tuturor. Fiecare membru pune în `contracts/` un exemplu complet al fișierului pe care îl produce, ca ceilalți să lucreze pe el până la integrare. Legătura dintre toate fișierele este `node_id`; legătura cu pagina reală este `selector`, generat de aceeași funcție `selector.js` folosită de toți.

nodes.json (Membrul 1)

```
{ "url": "https://...", "title": "Checkout", "viewport": {"width": 1280, "height": 720},
  "nodes": [
    { "id": "n12", "order": 12, "role": "button", "name": "", "level": null,
      "states": {"required": false, "checked": null, "expanded": null, "disabled": false},
      "value": null, "list_pos": null, "focusable": true, "parent": "n3",
      "selector": "#cart > button:nth-of-type(1)", "bbox": {"x": 1040, "y": 22, "w": 32, "h": 32} } ],
  "focus_order": ["n5", "n9", "n12"],
  "focus_trap": null }                      // sau {"node_id": "n40"} dacă Tab rămâne blocat
```

events.json (Membrul 2)

```
{ "lang": "ro", "events": [
    { "seq": 0, "node_id": "n1",  "text": "Titlu nivel 1, Finalizează comanda" },
    { "seq": 1, "node_id": "n12", "text": "Buton" } ] }
```

issues.json (Membrul 2)

```
[ { "id": "i1", "rule": "button-name", "severity": "critical", "node_id": "n12",
      "selector": "#cart > button:nth-of-type(1)", "title": "Butonul nu are nume accesibil",
      "recommendation": "Adaugă text vizibil sau aria-label care descrie acțiunea." } ]
```

timeline.json (Membrul 3)

```
{ "duration": 84.2, "clip": "clip.mp4",
  "entries": [ { "seq": 1, "node_id": "n12", "start": 41.8, "end": 42.6 } ] }
```

report.json (Membrul 2, consumat de API și interfață)

```
{ "id": "a7f3", "url": "https://...", "score": 54,
  "counts": {"critical": 2, "serious": 5, "moderate": 4, "minor": 1},
  "clip_url": "/media/a7f3/clip.mp4", "duration": 84.2,
  "issues": [ { ...issue, "timestamp": 41.8 } ] }        // timestamp null dacă nu apare în clip
```

Scorul: pornește de la 100 și scade per problemă: critical 10, serious 6, moderate 3, minor 1, cu maxim 20 de puncte pierdute pe aceeași regulă; capcana de tastatură scade încă 15. Minim 0.

7. Împărțirea sarcinilor

Fiecare membru are o bucată care se poate construi de la început până la capăt fără să aștepte pe nimeni: intrările vin din datele de test din `contracts/`, ieșirea trebuie să respecte contractul. La integrare, datele de test sunt înlocuite cu ieșirile reale ale celorlalți.

Membrul 1: Agentul și extracția

- Imaginea Docker a worker-ului: Python, Playwright + Chromium, Piper cu vocea română, ffmpeg.
- `extract(url)`: deschide pagina, așteaptă încărcarea, citește arborele cu `Accessibility.getFullAXTree`.
- Pentru fiecare nod: `DOM.resolveNode` → selector (cu `selector.js`) și `getBoundingClientRect`.
- Filtrează nodurile irelevante (generic, none, ascunse), păstrează ordinea de citire.
- Parcurgerea cu Tab: apasă Tab de maxim 80 de ori, notează elementul activ, detectează capcanele (același element de 3 ori la rând, fără să fi ajuns la final).
- Scrie `selector.js` în prima oră, pentru că îl folosesc și Membrii 2 și 3.

  Lucrează pe: URL-uri reale (2-3 pagini de test alese în prima oră).
  Gata când: nodes.json valid pe toate paginile de test; containerul worker pornește.

Membrul 2: Motorul de anunțuri, auditul și raportul

- `build_events(nodes)`: reguli pe rol (heading cu nivel, link, buton, câmp cu etichetă și stare, imagine cu sau fără descriere, element de listă cu poziție, landmark). Text static scurtat la 12 cuvinte, maxim 40 de evenimente.
- `run_audit(url, nodes)`: injectează axe-core, rulează, transformă ținta fiecărei probleme în selector canonic cu `selector.js` și o leagă de `node_id`.
- Șabloane de recomandare în română pentru cele mai frecvente ~15 reguli axe; restul folosesc textul de ajutor din axe.
- `build_report(nodes, issues, timeline)`: atașează secunda din clip, calculează scorul și sumarul.

  Lucrează pe: sample_nodes.json, sample_timeline.json; pentru audit, URL-urile de test.
  Gata când: events.json, issues.json, report.json corecte pe datele de test și pe paginile reale.

Membrul 3: Vocea și clipul

- TTS: fiecare `text` din events.json → WAV cu Piper; durata se citește din fișier.
- Înregistrarea: Playwright cu înregistrare video (1280×720), overlay injectat (contur colorat poziționat după bbox), scroll la element, ținut pe ecran cât durează propoziția + 0,3 s pauză.
- Notează momentul de start al fiecărui eveniment față de începutul înregistrării; taie din video perioada de încărcare.
- ffmpeg: pune fiecare WAV la momentul lui (adelay + amix), mixează peste video, exportă MP4 (H.264, AAC).
- `render_clip(url, nodes, events)` → clip.mp4 + timeline.json.

  Lucrează pe: sample_nodes.json + sample_events.json și pagina de test corespunzătoare.
  Gata când: Un MP4 sincronizat (vocea se aude exact când e evidențiat elementul) și timeline.json corect.

Membrul 4: API, interfața, orchestrarea și demo-ul

- FastAPI: `POST /api/analyses` {url} → {id}; `GET /api/analyses/{id}` → status + report; fișierele din `/media/{id}/` servite static.
- Statusuri: `queued → extracting → analyzing → rendering → done | failed`, cu mesaj de eroare.
- `pipeline.py`: job RQ care apelează etapele în ordine (la început pe stub-uri care copiază datele de test) și actualizează statusul.
- Next.js: pagina de start cu câmpul URL, pagina de progres, pagina de rezultat cu player, scor, listă de probleme filtrabilă pe severitate, click → salt la timestamp, buton de copiere link.
- Slide-urile, scenariul de prezentare și rezultatul de rezervă pentru demo.

  Lucrează pe: sample_report.json + un clip MP4 oarecare.
  Gata când: Fluxul complet merge cap-coadă pe stub-uri; interfața afișează orice report.json valid.

8. Programul

Când	Ce facem
Prima oră (toți)	Alegem 2-3 pagini de test (una cu probleme evidente). Stabilim contractele de date, scriem selector.js și datele de test din contracts/. Creăm repo-ul cu structura de foldere și docker-compose.
Dezvoltare	Fiecare lucrează pe bucata lui, pe date de test. Commit-uri mici pe branch propriu.
Checkpoint la jumătate	Prima integrare reală: extract → build_events → render_clip pe o singură pagină. Fiecare vede dacă ieșirea lui merge la următorul.
Ultimele 4 ore	Integrare completă, stub-urile din pipeline.py înlocuite cu codul real, reparat ce nu se leagă.
Ultimele 2 ore	Generăm dinainte rezultatul pentru site-ul de demo (backup dacă pică internetul), repetăm prezentarea. Nu se mai adaugă funcționalități.

> **Regula de aur:** dacă trebuie schimbat un contract de date, se anunță toată echipa și se actualizează exemplul din contracts/ înainte de orice alt cod. Restul deciziilor din bucata ta le iei singur.